

机器人队 C++ 面向对象与 CMake 入门

这不是一份“背语法”的教程。我们会让同一个电机控制程序一步步长大：普通变量 → **Motor** → 统一接口 → 不同品牌电机 → **Robot** → 多文件工程 → CMake。每个目录中的程序都已经可以独立编译。

Chapter 00：Ubuntu 开发环境配置

本章目标

在刚安装好的 Ubuntu Desktop 上装好本教程需要的工具，并建立统一工作目录。

操作

按 **Ctrl + Alt + T** 打开终端。先更新软件包列表：

```
sudo apt update
```

sudo 表示“以管理员身份执行”；它会要求输入当前用户密码。输入密码时屏幕通常不显示 * 或任何字符，这是正常现象，直接输入后按 Enter。**apt** 是 Ubuntu 的软件包工具，**update** 是刷新可安装软件的信息，并不等于升级系统。

安装工具：

```
sudo apt install -y build-essential cmake git tree nano
```

build-essential 提供 **gcc**、**g++**、**make** 等基础编译工具；**cmake** 用于构建工程；**git** 是队伍今后常用的版本管理工具（本教程暂不展开）；**tree** 用树状方式查看目录；**nano** 是终端文本编辑器。**-y** 表示自动确认安装。

逐条验证。只要出现版本号即可，不要求与本讲义完全相同：

```
g++ --version
cmake --version
make --version
git --version
```

建立所有练习的工作目录：

```
cd ~
mkdir -p robot_cpp_training
cd robot_cpp_training
pwd
```

~ 是当前用户的家目录；`mkdir` 创建目录，`-p` 允许父目录已存在；`cd` 切换目录；`pwd` 显示当前位置。

本教程会用到的最少 Linux 命令：`ls` 列出文件，`cd 目录名` 进入目录，`cd ..` 回到上一级，`mkdir 名字` 新建目录，`cp 源 目标` 复制文件，`rm 文件名` 删除一个文件。删除不可恢复；尤其不要随意对不确定的目录使用递归删除。

创建空的培训工程根目录

```
cd ~/robot_cpp_training
mkdir -p robot_cpp_oop_cmake_training
cd robot_cpp_oop_cmake_training
pwd
```

此时我们只是创建了一个空的培训工程根目录。路径类似：

```
/home/用户名/robot_cpp_training/robot_cpp_oop_cmake_training
```

从现在开始，我们不会下载一个已经写好的工程，而是按照讲义，从零亲手创建整个 C++ 项目。

可选：老师提供完整仓库时

老师可能额外提供一个供课后对照的完整仓库：

```
git clone <仓库地址>
```

`git clone` 的意思是把远程仓库下载到本地。这不是本教程主流程；即使没有仓库，只靠本讲义也可以完成全部实验。

第一次创建和编辑代码文件

本教程不依赖 IDE。即使是只有终端的全新 Ubuntu，也可用 `nano` 完成全部实验：

```
mkdir -p ~/robot_cpp_training/test
cd ~/robot_cpp_training/test
nano main.cpp
```

在打开的编辑器中输入代码后，按 `Ctrl + O` 保存，按 `Enter` 确认文件名，再按 `Ctrl + X` 退出。后续讲义中的代码既可用 `nano` 创建，也可用 VS Code、CLion 等编辑器；若系统没有 `nano`，执行 `sudo apt install -y nano`。

常见错误

`command not found` 通常是工具尚未安装或命令拼错；回到本章重新安装。`Permission denied` 时先确认自己是否正在操作需要管理员权限的系统目录；教程目录不需要 `sudo`。

Chapter 01：第一个 C++ 程序

本章目标与问题

先让机器人在终端“开口说话”。从空工程根目录开始创建本章目录和文件：

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p examples/01_hello_cpp
cd examples/01_hello_cpp
nano main.cpp
```

输入完整代码：

```
#include <iostream>

int main()
{
    std::cout << "Hello Robot!" << std::endl;
    return 0;
}
```

复制完成后按 **Ctrl + O** 保存，按 **Enter** 确认文件名，再按 **Ctrl + X** 退出。然后编译运行：

```
g++ main.cpp -std=c++17 -o demo
./demo
```

预期输出：

```
Hello Robot!
```

代码解释与核心知识点

.cpp 是 C++ 源代码文件。**#include <iostream>** 引入终端输入输出工具；**int main()** 是程序入口；**{}** 包住函数内容；**std::cout** 向终端输出；**std::endl** 换行；每条普通 C++ 语句以 **;** 结束；**return 0** 表示正常结束。

```
源代码 main.cpp → g++ → 可执行程序 demo → ./demo 运行
```

不要写 **using namespace std;**；从一开始写 **std::cout**，能避免大型工程中名称冲突。

常见错误与练习

`./demo: No such file` 表示没有成功编译或当前目录不对，先执行 `pwd`、`ls`。忘记 `;` 时编译器会给出行号，先从那一行和前一检查。练习：把文字改为 `Hello Robot Team!`，重新编译再运行。

Chapter 02：从普通变量到 Motor 类

为什么需要

最开始两台电机可能写成 `motor1_id`、`motor1_position`。再加速度、力矩、温度后，12 台电机会有很长一串彼此相关又容易写错的变量。把“一台电机的数据”放进 `Motor`，程序的结构就清楚了。

完整代码：`examples/02_basic_motor/main.cpp`

从稳定的根目录开始，创建本章文件：

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p examples/02_basic_motor
cd examples/02_basic_motor
nano main.cpp
```

```
#include <iostream>

class Motor {
public:
    int id;
    double position;
    double velocity;
    double torque;
    double temperature;
};

int main()
{
    Motor motor1;
    motor1.id = 1;
    motor1.position = 0.0;
    motor1.velocity = 0.0;
    motor1.torque = 0.0;
    motor1.temperature = 25.0;

    Motor motor2;
    motor2.id = 2;
    motor2.position = 0.5;
    motor2.velocity = 0.1;
    motor2.torque = 1.2;
    motor2.temperature = 26.5;

    std::cout << "Motor " << motor1.id << " position: " << motor1.position
    << " rad" << std::endl;
    std::cout << "Motor " << motor2.id << " position: " << motor2.position
    << " rad" << std::endl;
```

```
    return 0;  
}
```

保存并退出 nano 后执行：

```
g++ main.cpp -std=c++17 -o demo  
./demo
```

预期输出包含 `Motor 1 position: 0 rad` 与 `Motor 2 position: 0.5 rad`。`class Motor` 是蓝图，`motor1`、`motor2` 是按蓝图创建的对象；其中的 `id` 等是成员变量。当前它们全在 `public`，任何代码都能随意修改，这正是下一章要解决的问题。练习：创建 `Motor motor3` 并输出其温度。

Chapter 03：成员函数、public/private 与封装

为什么需要

直接写 `motor.position = 1000` 会绕过角度限制，真实电机可能因此收到危险命令。让外部通过函数设置值，类就能统一执行保护。

完整代码：`examples/03_encapsulation/main.cpp`

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training  
mkdir -p examples/03_encapsulation  
cd examples/03_encapsulation  
nano main.cpp
```

```
#include <iostream>  
  
class Motor {  
public:  
    void setPosition(double position)  
    {  
        if (position > 3.14) position = 3.14;  
        if (position < -3.14) position = -3.14;  
        position_ = position;  
    }  
    double getPosition() const { return position_; }  
    void printStatus() const  
    {  
        std::cout << "Motor position: " << position_ << " rad" <<  
std::endl;  
    }  
private:  
    double position_ = 0.0;  
};
```

```
int main()
{
    Motor motor;
    motor.setPosition(4.0);
    motor.printStatus();
    std::cout << "Read by getter: " << motor.getPosition() << " rad" <<
std::endl;
    return 0;
}
```

保存并退出 nano 后执行：

```
g++ main.cpp -std=c++17 -o demo
./demo
```

预期位置为 3.14 rad，因为已限幅。public 的成员可以被外部调用，private 只能由本类使用；setPosition 是 setter，getPosition 是 getter。函数后的 const 承诺它不修改对象状态。把数据与保护逻辑包在一起叫封装；实际还能放角度/速度限幅、零点补偿、单位转换、安全检查和 CAN 报文发送。

为了突出封装概念，这里直接写 3.14。实际机器人项目通常不会把限位直接写死在函数里，而会使用常量、配置文件或设备参数，例如 min_position、max_position；后续项目应尽量做到“参数配置化”。本章暂不引入 YAML 或配置解析。练习：把限幅改为 [-2.0, 2.0]。

Chapter 04：构造函数与对象初始化

为什么需要

每次创建对象再单独设置 ID 容易遗漏。构造函数让对象诞生时就处在有效状态。

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p examples/04_constructor
cd examples/04_constructor
nano main.cpp
```

```
#include <iostream>

class Motor {
public:
    Motor(int id) : id_(id), position_(0.0) {}
    void setPosition(double position) { position_ = position; }
    void printStatus() const
    {
        std::cout << "Motor " << id_ << " position: " << position_ << "
rad" << std::endl;
    }
private:
```

```

    int id_;
    double position_;
};

int main()
{
    Motor motor(1);
    motor.setPosition(1.25);
    motor.printStatus();
    return 0;
}

```

保存并退出 nano 后执行：

```

g++ main.cpp -std=c++17 -o demo
./demo

```

预期输出 `Motor 1 position: 1.25 rad`。`Motor motor(1)` 调用 `Motor(int id)`；冒号后的初始化列表在函数体执行前初始化成员。`id` 是构造函数的局部参数，`id_` 是成员变量；末尾下划线是本工程用来区分成员变量的约定。`this` 可理解为“当前对象”，本章通常无需显式使用它。练习：构造函数再接收一个初始位置。

Chapter 05：继承前为什么需要统一接口

为什么需要

达妙和宇树电机的通信不同。如果上层写 `setDMPosition()`、`setUnitreePosition()`，控制器会被具体型号绑死。先规定所有电机都必须有什么能力。

```

cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p examples/05_abstract_motor
cd examples/05_abstract_motor
nano main.cpp

```

```

#include <iostream>

class Motor {
public:
    virtual void enable() = 0;
    virtual void setPosition(double position) = 0;
    virtual double getPosition() const = 0;
    virtual ~Motor() = default;
};

class SimulatedMotor : public Motor {
public:
    void enable() override

```

```

    {
        enabled_ = true;
        std::cout << "Simulated motor enabled." << std::endl;
    }

    void setPosition(double position) override
    {
        position_ = position;
        std::cout << "Simulated target: " << position_ << " rad" <<
std::endl;
    }

    double getPosition() const override
    {
        return position_;
    }
private:
    bool enabled_ = false;
    double position_ = 0.0;
};

int main()
{
    SimulatedMotor motor;
    motor.enable();
    motor.setPosition(1.0);
    std::cout << "Position: " << motor.getPosition() << " rad" <<
std::endl;
    return 0;
}

```

保存并退出 nano 后执行：

```

g++ main.cpp -std=c++17 -o demo
./demo

```

virtual 表示允许子类提供自己的实现；**= 0** 是纯虚函数，只有规定、没有实现。含纯虚函数的 **Motor** 是抽象类，不能直接 **Motor motor**；。它描述的是“一台电机应该提供什么能力”。**Motor** 是一个准备被多态使用的父类，因此这里给它一个虚析构函数；第一阶段先把它当作“多态基类的推荐写法”记住即可。等以后学习动态内存、RAII、智能指针时，再深入析构函数的意义。练习：尝试直接创建 **Motor**，阅读编译错误。

Chapter 06：DMMotor 与 UnitreeMotor——继承与 override

完整代码：[examples/06_inheritance/main.cpp](#)

```

cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p examples/06_inheritance

```

```
cd examples/06_inheritance
nano main.cpp
```

```
#include <iomanip>
#include <iostream>

class Motor {
public:
    virtual void enable() = 0;
    virtual void setPosition(double position) = 0;
    virtual double getPosition() const = 0;
    virtual ~Motor() = default;
};

class DMMotor : public Motor {
public:
    explicit DMMotor(int id) : id_(id) {}
    void enable() override { std::cout << "DM Motor " << id_ << " enabled."
<< std::endl; }
    void setPosition(double position) override
    {
        position_ = position;
        std::cout << "DM Motor " << id_ << " -> target = " << std::fixed <<
std::setprecision(2) << position_ << " rad" << std::endl;
    }
    double getPosition() const override { return position_; }
private:
    int id_;
    double position_ = 0.0;
};

class UnitreeMotor : public Motor {
public:
    explicit UnitreeMotor(int id) : id_(id) {}
    void enable() override { std::cout << "Unitree Motor " << id_ << "
enabled." << std::endl; }
    void setPosition(double position) override
    {
        position_ = position;
        std::cout << "Unitree Motor " << id_ << " -> target = " <<
std::fixed << std::setprecision(2) << position_ << " rad" << std::endl;
    }
    double getPosition() const override { return position_; }
private:
    int id_;
    double position_ = 0.0;
};

int main()
{
    DMMotor dm_motor(1);
```

```

UnitreeMotor unitree_motor(2);
dm_motor.enable();
unitree_motor.enable();
dm_motor.setPosition(1.5);
unitree_motor.setPosition(1.5);
return 0;
}

```

保存并退出 nano 后执行：

```

g++ main.cpp -std=c++17 -o demo
./demo

```

预期输出：

```

DM Motor 1 enabled.
Unitree Motor 2 enabled.
DM Motor 1 -> target = 1.50 rad
Unitree Motor 2 -> target = 1.50 rad

```

父类是 `Motor`，子类是两种具体电机；`public Motor` 表示“DMMotor is a Motor”，这就是继承。`override` 让编译器检查函数确实覆盖了父类接口，拼写或 `const` 写错时能尽早报错。`std::fixed << std::setprecision(2)` 只是让浮点输出稳定显示两位小数。

练习：阅读该文件，给两个 `setPosition` 的输出增加不同的协议提示文字。

Chapter 07：多态

完整代码：`examples/07_polymorphism/main.cpp`

```

cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p examples/07_polymorphism
cd examples/07_polymorphism
nano main.cpp

```

```

#include <iomanip>
#include <iostream>

class Motor {
public:
    virtual void enable() = 0;
    virtual void setPosition(double position) = 0;
    virtual double getPosition() const = 0;
    virtual ~Motor() = default;

```

```
};

class DMMotor : public Motor {
public:
    explicit DMMotor(int id) : id_(id) {}
    void enable() override { std::cout << "DM Motor " << id_ << " enabled."
<< std::endl; }
    void setPosition(double position) override { position_ = position;
std::cout << "DM Motor " << id_ << " -> target = " << std::fixed <<
std::setprecision(2) << position_ << " rad" << std::endl; }
    double getPosition() const override { return position_; }
private:
    int id_;
    double position_ = 0.0;
};

class UnitreeMotor : public Motor {
public:
    explicit UnitreeMotor(int id) : id_(id) {}
    void enable() override { std::cout << "Unitree Motor " << id_ << "
enabled." << std::endl; }
    void setPosition(double position) override { position_ = position;
std::cout << "Unitree Motor " << id_ << " -> target = " << std::fixed <<
std::setprecision(2) << position_ << " rad" << std::endl; }
    double getPosition() const override { return position_; }
private:
    int id_;
    double position_ = 0.0;
};

void controlMotor(Motor& motor)
{
    motor.enable();
    motor.setPosition(1.5);
}

int main()
{
    DMMotor dm_motor(1);
    UnitreeMotor unitree_motor(2);
    controlMotor(dm_motor);
    controlMotor(unitree_motor);
    return 0;
}
```

保存并退出 nano 后执行：

```
g++ main.cpp -std=c++17 -o demo
./demo
```

预期输出：

```
DM Motor 1 enabled.
DM Motor 1 -> target = 1.50 rad
Unitree Motor 2 enabled.
Unitree Motor 2 -> target = 1.50 rad
```

`controlMotor(dm_motor);` 和 `controlMotor(unitree_motor);` 调用的是同一个函数。它根本不知道具体型号，但同一句 `motor.setPosition()` 执行了不同实现：这就是多态，即“相同接口，不同实现”。
`Motor&` 是引用，可以把已有对象交给函数而不复制，也保留实际对象的类型信息。本章不需要先学习复杂指针。练习：把目标改为 `-0.8`。

Chapter 08: Robot 类与组合

真实机器人的结构不是“机器人是一台电机”，而是“机器人使用电机”。先把这种 `has-a` 关系理解为对象组合思想：

```
Robot
├── Motor
├── Motor
├── IMU
├── Controller
└── Camera
```

本教程只实现两个电机。完整代码：`examples/08_robot_composition/main.cpp`

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p examples/08_robot_composition
cd examples/08_robot_composition
nano main.cpp
```

```
#include <iomanip>
#include <iostream>

class Motor {
public:
    virtual void enable() = 0;
    virtual void setPosition(double position) = 0;
    virtual double getPosition() const = 0;
    virtual ~Motor() = default;
};

class DMMotor : public Motor {
public:
    explicit DMMotor(int id) : id_(id) {}
```

```

    void enable() override { std::cout << "DM Motor " << id_ << " enabled."
<< std::endl; }
    void setPosition(double position) override { position_ = position;
std::cout << "DM Motor " << id_ << " -> target = " << std::fixed <<
std::setprecision(2) << position_ << " rad" << std::endl; }
    double getPosition() const override { return position_; }
private:
    int id_;
    double position_ = 0.0;
};

class UnitreeMotor : public Motor {
public:
    explicit UnitreeMotor(int id) : id_(id) {}
    void enable() override { std::cout << "Unitree Motor " << id_ << "
enabled." << std::endl; }
    void setPosition(double position) override { position_ = position;
std::cout << "Unitree Motor " << id_ << " -> target = " << std::fixed <<
std::setprecision(2) << position_ << " rad" << std::endl; }
    double getPosition() const override { return position_; }
private:
    int id_;
    double position_ = 0.0;
};

class Robot {
public:
    Robot(Motor& left_motor, Motor& right_motor)
        : left_motor_(left_motor), right_motor_(right_motor) {}
    void initialize() { std::cout << "Robot initialization..." <<
std::endl; left_motor_.enable(); right_motor_.enable(); }
    void move(double position) { std::cout << "Set robot target position: "
<< std::fixed << std::setprecision(2) << position << " rad" << std::endl;
left_motor_.setPosition(position); right_motor_.setPosition(position); }
private:
    Motor& left_motor_;
    Motor& right_motor_;
};

int main()
{
    DMMotor left_motor(1);
    UnitreeMotor right_motor(2);
    Robot robot(left_motor, right_motor);
    robot.initialize();
    robot.move(1.5);
    return 0;
}

```

保存并退出 nano 后执行：

```
g++ main.cpp -std=c++17 -o demo
./demo
```

预期输出：

```
Robot initialization...
DM Motor 1 enabled.
Unitree Motor 2 enabled.
Set robot target position: 1.50 rad
DM Motor 1 -> target = 1.50 rad
Unitree Motor 2 -> target = 1.50 rad
```

构造时传入两个满足 **Motor** 接口的对象，`initialize()` 只管使能，`move()` 只管发送位置。**DMMotor is a Motor** 是继承；**Robot has Motors** 是对象组合思想。

从软件设计角度，可以先把本例理解成 **Robot has Motors** 的 has-a 关系。但这里 **Robot** 保存的是外部 **Motor** 对象的引用，因此严格来说属于非拥有型关联/聚合关系，并不拥有电机的生命周期。真正由 **Robot** 自己创建并管理生命周期的对象，才更接近严格意义上的 **composition**。第一阶段只需要理解继承与对象组合思想；更严格的生命周期和所有权问题以后学习智能指针时再展开。练习：让左右目标位置分别不同（需要修改 `move` 的参数）。

Chapter 09：为什么需要 **.hpp** 和 **.cpp**

到这里若把 **Robot**、**Motor**、IMU、Controller 和 `main` 都塞进一个文件，维护会变困难。拆分后的直观分工是：

```
.hpp：告诉别人“我有什么”（声明、接口、类定义）
.cpp：告诉别人“我是怎么做的”（函数具体实现）
```

最终工程使用 `include/` 存放可被其他代码包含的头文件，`src/` 存放实现，`apps/` 存放可执行程序入口。头文件的 `#pragma once` 防止同一文件被重复包含。

下一章我们将不再只创建一个 `main.cpp`，而是亲手建立真实的多文件工程。

Chapter 10：正式建立 **mini_robot** 工程

先在项目根目录创建最终工程目录：

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p mini_robot/include/robot
mkdir -p mini_robot/src
mkdir -p mini_robot/apps
cd mini_robot
tree
```

此时目录还是空的，预期为：

```
.
├── apps
├── include
│   └── robot
└── src
```

其命名空间统一为 `namespace robot`；因此 `main.cpp` 写 `robot::DMMotor`。命名空间避免以后队内不同模块都有 `Robot` 或 `Motor` 时发生名称冲突。头文件用 `#include "robot/motor.hpp"`，由后续的 `-Iinclude` 或 CMake include 路径找到。

文件 1: `include/robot/motor.hpp`

```
nano include/robot/motor.hpp
```

```
#pragma once

namespace robot {

class Motor {
public:
    virtual void enable() = 0;
    virtual void setPosition(double position) = 0;
    virtual double getPosition() const = 0;
    virtual ~Motor() = default;
};

} // namespace robot
```

`#pragma once` 防止头文件被重复包含；`namespace robot` 是命名空间；这里的 `Motor` 是抽象类，三个 `= 0` 的函数是纯虚函数。

文件 2: `include/robot/dm_motor.hpp`

```
nano include/robot/dm_motor.hpp
```

```
#pragma once

#include "robot/motor.hpp"

namespace robot {
```

```

class DMMotor : public Motor {
public:
    explicit DMMotor(int id);

    void enable() override;
    void setPosition(double position) override;
    double getPosition() const override;

private:
    int id_;
    double position_ = 0.0;
};

} // namespace robot

```

`#include "robot/motor.hpp"` 使用我们自己的头文件；`public Motor` 表示继承；`explicit` 防止 ID 被意外转换构造造成对象；`override` 检查覆盖接口；ID 和位置放在 `private` 中。

文件 3: `include/robot/unitree_motor.hpp`

```
nano include/robot/unitree_motor.hpp
```

```

#pragma once

#include "robot/motor.hpp"

namespace robot {

class UnitreeMotor : public Motor {
public:
    explicit UnitreeMotor(int id);

    void enable() override;
    void setPosition(double position) override;
    double getPosition() const override;

private:
    int id_;
    double position_ = 0.0;
};

} // namespace robot

```

文件 4: `include/robot/robot.hpp`

```
nano include/robot/robot.hpp
```

```
#pragma once

#include "robot/motor.hpp"

namespace robot {

class Robot {
public:
    Robot(Motor& left_motor, Motor& right_motor);
    void initialize();
    void move(double position);
    void printStatus() const;
private:
    Motor& left_motor_;
    Motor& right_motor_;
};

} // namespace robot
```

`Motor&` 是引用：`Robot` 使用传进来的电机对象而不复制它。这里不负责其生命周期。

文件 5: `src/dm_motor.cpp`

```
nano src/dm_motor.cpp
```

```
#include "robot/dm_motor.hpp"

#include <iomanip>
#include <iostream>

namespace robot {

DMMotor::DMMotor(int id)
    : id_(id)
{
}

void DMMotor::enable()
{
    std::cout << "DM Motor " << id_ << " enabled." << std::endl;
}

void DMMotor::setPosition(double position)
{
}
```

```

        position_ = position;
        std::cout << "DM Motor " << id_ << " -> target = " << std::fixed
                    << std::setprecision(2) << position_ << " rad" << std::endl;
    }

    double DMMotor::getPosition() const
    {
        return position_;
    }

} // namespace robot

```

`DMMotor::DMMotor` 是构造函数，`DMMotor::enable` 等是成员函数实现。`::` 是作用域解析符，表示“这个函数属于 `DMMotor`”。

文件 6: `src/unitree_motor.cpp`

```
nano src/unitree_motor.cpp
```

```

#include "robot/unitree_motor.hpp"

#include <iomanip>
#include <iostream>

namespace robot {

UnitreeMotor::UnitreeMotor(int id)
    : id_(id)
{
}

void UnitreeMotor::enable()
{
    std::cout << "Unitree Motor " << id_ << " enabled." << std::endl;
}

void UnitreeMotor::setPosition(double position)
{
    position_ = position;
    std::cout << "Unitree Motor " << id_ << " -> target = " << std::fixed
                << std::setprecision(2) << position_ << " rad" << std::endl;
}

double UnitreeMotor::getPosition() const
{
    return position_;
}

}

```

```
} // namespace robot
```

文件 7: `src/robot.cpp`

```
nano src/robot.cpp
```

```
#include "robot/robot.hpp"

#include <iomanip>
#include <iostream>

namespace robot {

Robot::Robot(Motor& left_motor, Motor& right_motor)
    : left_motor_(left_motor), right_motor_(right_motor)
{
}

void Robot::initialize()
{
    std::cout << "Robot initialization..." << std::endl << std::endl;
    left_motor_.enable();
    right_motor_.enable();
}

void Robot::move(double position)
{
    std::cout << std::endl << "Set robot target position: " << std::fixed
        << std::setprecision(2) << position << " rad" << std::endl <<
std::endl;
    left_motor_.setPosition(position);
    right_motor_.setPosition(position);
    std::cout << std::endl << "Robot motion command finished." <<
std::endl;
}

void Robot::printStatus() const
{
    std::cout << std::endl << "Left motor position: " << std::fixed
        << std::setprecision(2) << left_motor_.getPosition() << "
rad" << std::endl;
    std::cout << "Right motor position: " << std::fixed
        << std::setprecision(2) << right_motor_.getPosition() << "
rad" << std::endl;
}

} // namespace robot
```

文件 8: apps/main.cpp

```
nano apps/main.cpp
```

```
#include "robot/dm_motor.hpp"
#include "robot/robot.hpp"
#include "robot/unitree_motor.hpp"

#include <iostream>

int main()
{
    std::cout << "=== Mini Robot Demo ===" << std::endl << std::endl;

    robot::DMMotor left_motor(1);
    robot::UnitreeMotor right_motor(2);
    robot::Robot robot(left_motor, right_motor);

    robot.initialize();
    robot.move(1.5);
    robot.printStatus();
    return 0;
}
```

完成后执行:

```
tree
```

预期至少看到:

```
.
├── apps
│   └── main.cpp
├── include
│   └── robot
│       ├── dm_motor.hpp
│       ├── motor.hpp
│       ├── robot.hpp
│       └── unitree_motor.hpp
└── src
    ├── dm_motor.cpp
    ├── robot.cpp
    └── unitree_motor.cpp
```

此时 `CMakeLists.txt` 还没有创建。这是故意的：下一步先不用 CMake，而是手动调用 `g++`，帮助理解多文件编译。

Chapter 11：先不用 CMake，使用 `g++` 手动编译

先进入最终工程：

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training/mini_robot
g++ \
  apps/main.cpp \
  src/dm_motor.cpp \
  src/unitree_motor.cpp \
  src/robot.cpp \
  -Iinclude \
  -std=c++17 \
  -Wall \
  -Wextra \
  -pedantic \
  -o robot_demo
./robot_demo
```

`g++` 是编译器；四个 `.cpp` 是所有源文件；`-Iinclude` 添加头文件搜索目录；`-std=c++17` 选择本教程标准；`-o robot_demo` 指定输出可执行文件名。`-Wall`、`-Wextra`、`-pedantic` 开启更多编译器检查，帮助发现问题。少写任意一个实现 `.cpp` 常会在最后出现 `undefined reference`。

Chapter 12：编译与链接

手动命令大致做了下面的工作：

```
main.cpp → compile → main.o
dm_motor.cpp → compile → dm_motor.o
unitree_motor.cpp → compile → unitree_motor.o
robot.cpp → compile → robot.o
所有 .o → link → robot_demo
```

源文件是 `.cpp`；编译产生 object file（目标文件）`.o`；链接把它们合成 executable（可执行程序）。可亲手只编译一个实现：

```
g++ -c src/dm_motor.cpp -Iinclude -std=c++17 -o dm_motor.o
ls dm_motor.o
```

`-c` 表示只编译、不链接。练习后可以用 `rm dm_motor.o` 删除这个临时文件。

Chapter 13：为什么需要 CMake

4 个 `.cpp` 还可以手写，20、50、100 个文件就容易漏写。CMake 是构建系统的生成器，**不是编译器**：

```
CMakeLists.txt → CMake → Makefile / Ninja rules → build tool → g++ →  
executable / library
```

它把“有哪些文件、谁依赖谁、头文件在哪里”写成可维护的工程描述。

Chapter 14: 第一个 CMakeLists.txt

先创建这个新文件：

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training/mini_robot  
nano CMakeLists.txt
```

输入最简单的可执行程序版本，保存并退出：

```
cmake_minimum_required(VERSION 3.16)  
project(mini_robot)  
set(CMAKE_CXX_STANDARD 17)  
set(CMAKE_CXX_STANDARD_REQUIRED ON)  
add_executable(robot_demo apps/main.cpp src/dm_motor.cpp  
src/unitree_motor.cpp src/robot.cpp)  
target_include_directories(robot_demo PRIVATE include)
```

`cmake_minimum_required` 限定最低 CMake；`project` 命名工程；`set` 设置 C++17；`add_executable` 定义一个可执行 target；`target_include_directories` 给该 target 添加头文件目录；`PRIVATE` 表示该设置只由这个 target 自己使用。

Chapter 15: 标准 CMake 构建流程

最终工程采用推荐的 out-of-source build：

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training/mini_robot  
cmake -S . -B build  
cmake --build build  
./build/robot_demo  
tree -L 2
```

`-S .` 指源码目录是当前目录；`-B build` 指构建产物放在 `build`。这样 `src`、`include`、`apps` 保持干净，生成的缓存、目标文件和可执行程序都进入 `build`。

Chapter 16: CMake 的 target 思想

现在我们把第一版 `CMakeLists.txt` 升级为“核心库 + 可执行程序”结构。再次打开：

```
nano CMakeLists.txt
```

删除原来的全部内容，替换为：

```
cmake_minimum_required(VERSION 3.16)
project(mini_robot)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_library(robot_core
    src/dm_motor.cpp
    src/unitree_motor.cpp
    src/robot.cpp
)
target_include_directories(robot_core PUBLIC include)

add_executable(robot_demo apps/main.cpp)
target_link_libraries(robot_demo PRIVATE robot_core)
```

现代 CMake 的核心是 target：设置尽量挂在对应 target 上，而不是全局变量。`PUBLIC include` 表示 `robot_core` 自己需要 `include`，链接它的 `robot_demo` 也需要；`target_link_libraries` 声明 `demo` 依赖 `core`。

修改 `CMakeLists.txt` 后重新执行 `cmake -S . -B build` 即可让 CMake 更新构建规则。若真的需要完全重新配置，可以执行 `rm -rf build`，但它有风险，只能在确认当前路径正确时使用。

```
cmake -S . -B build
cmake --build build
./build/robot_demo
```

Chapter 17: library 与 executable

```
dm_motor.cpp, unitree_motor.cpp, robot.cpp
    ↓
    robot_core (library)
    ↓ link
main.cpp → robot_demo (executable)
```

library 是可被多个程序复用的一组已编译代码，本身通常不直接运行；executable 有 `main()`，可以直接运行。未来 ROS 2 节点、算法库也常遵循“核心库 + 若干应用”的结构。

Chapter 18: 最终运行效果

运行 `./build/robot_demo` 预期得到：

```
=== Mini Robot Demo ===

Robot initialization...

DM Motor 1 enabled.
Unitree Motor 2 enabled.

Set robot target position: 1.50 rad

DM Motor 1 -> target = 1.50 rad
Unitree Motor 2 -> target = 1.50 rad

Robot motion command finished.

Left motor position: 1.50 rad
Right motor position: 1.50 rad
```

这是通信行为的终端模拟，不会向真实电机发送命令。

Chapter 19：最终知识回顾

C++ 基础：`class` 是蓝图，`object` 是对象；`public/private` 控制访问；`constructor` 负责初始化；`member variable/function` 组织状态和行为；`const` 表示不改状态；`reference` 用于引用已有对象；`namespace` 避免名称冲突。

面向对象：封装保护电机状态；抽象 `Motor` 描述能力；继承体现 `DMMotor is a Motor`；`virtual/override` 与多态让统一控制函数适配不同驱动；组合体现 `Robot has Motors`。

工程结构：`include` 放接口，`src` 放实现，`apps` 放入口，`build` 放构建产物。编译链路为 `.cpp → compiler → .o → linker → executable`。CMake 常用词是 `project()`、`add_library()`、`add_executable()`、`target_include_directories()`、`target_link_libraries()`。

Chapter 20：课后任务与排错

Task 1：在 `mini_robot/apps/main.cpp` 将 `robot.move(1.5);` 改成 `robot.move(2.0);`，重新执行 `cmake --build build` 和 `./build/robot_demo`。

Task 2：给 `Motor` 增加 `virtual void disable() = 0;`，让 `DM` 与 `Unitree` 都实现它；再给 `Robot` 增加 `shutdown()`，调用两台电机的 `disable()`。每次改动后重新构建。

Task 3：新增 `DJIMotor`：`public Motor`，实现 `enable`、`disable`、`setPosition`、`getPosition`，并把它作为 `Robot` 的一台电机。重点是：除了创建对象，尽可能不改 `Robot` 的控制逻辑。这正是抽象、继承、多态带来的价值。

常见错误：

- `command not found`：检查安装步骤和命令拼写。
- `No such file or directory`：执行 `pwd`、`ls`，确认当前目录和文件名。

- `expected ';'` : 通常漏了分号, 检查报错行及其前一行。
- `robot/motor.hpp: No such file:` 手动编译漏了 `-Iinclude`, 或在错误目录执行。
- `undefined reference:` 声明找到了、实现却没有参与链接; 检查是否漏掉某个 `.cpp` 或 `target_link_libraries`。
- CMake 提示找不到文件: 检查 `CMakeLists.txt` 内路径相对工程根目录是否正确、文件大小写是否一致。
- 修改后没变化: 先重新编译; CMake 工程运行的是 `./build/robot_demo`, 不是旧的 `./robot_demo`。

最终自动验证:

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
bash scripts/test_all.sh
```

下一阶段可以学习 STL (`std::vector`、`std::array`、`std::string`、`std::map`)、内存管理 (RAII、`std::unique_ptr`、`std::shared_ptr`)，再进入 Eigen、ROS 2、OpenCV、CAN 与串口。先保持本课程的主线：让结构清晰、让程序可编译、让每一步可验证。

Chapter 21: 自动测试整个教程

到这里，我们已经亲手创建了 8 个教学程序和一个 `mini_robot` 工程。最后写一个简单脚本，自动检查所有内容是否还能正常编译。它不依赖你当前在哪个目录执行。

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
mkdir -p scripts
nano scripts/test_all.sh
```

输入完整脚本:

```
#!/usr/bin/env bash

set -e

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

for example_name in \
    01_hello_cpp \
    02_basic_motor \
    03_encapsulation \
    04_constructor \
    05_abstract_motor \
    06_inheritance \
    07_polymorphism \
    08_robot_composition; do
    example_dir="$ROOT_DIR/examples/$example_name"
    (
        cd "$example_dir"
```

```

        g++ main.cpp \
            -std=c++17 \
            -Wall \
            -Wextra \
            -pedantic \
            -o demo
        ./demo
    )
    echo "[PASS] $example_name"
done

(
    cd "$ROOT_DIR/mini_robot"
    g++ \
        apps/main.cpp \
        src/dm_motor.cpp \
        src/unitree_motor.cpp \
        src/robot.cpp \
        -Iinclude \
        -std=c++17 \
        -Wall \
        -Wextra \
        -pedantic \
        -o robot_demo_manual
    ./robot_demo_manual
)
echo "[PASS] mini_robot manual g++"

(
    cd "$ROOT_DIR/mini_robot"
    cmake -S . -B build_test
    cmake --build build_test
    ./build_test/robot_demo
)
echo "[PASS] mini_robot CMake"

echo
echo "All examples passed."

```

`#!/usr/bin/env bash` 表示用 **Bash** 运行脚本，`set -e` 表示任意命令失败时立即停止。`ROOT_DIR` 自动计算项目根目录；每个圆括号 `(...)` 创建一个临时的命令环境，因此 `cd` 不会影响脚本的其他部分。

`build_test` 是测试专用构建目录，不会影响 Chapter 15 创建的 `build`。

保存脚本后执行：

```

chmod +x scripts/test_all.sh
bash scripts/test_all.sh

```

`chmod +x` 给脚本增加可执行权限。之后也可用 `./scripts/test_all.sh` 运行。预期输出会在最后的“最终验收”章节中汇总。

附录：mini_robot 全部可复制源码

本附录集中列出 Chapter 10 的逐文件完整源码，方便复查。

include/robot/motor.hpp

```
#pragma once

namespace robot {

class Motor {
public:
    virtual void enable() = 0;
    virtual void setPosition(double position) = 0;
    virtual double getPosition() const = 0;
    virtual ~Motor() = default;
};

} // namespace robot
```

include/robot/dm_motor.hpp

```
#pragma once

#include "robot/motor.hpp"

namespace robot {

class DMMotor : public Motor {
public:
    explicit DMMotor(int id);

    void enable() override;
    void setPosition(double position) override;
    double getPosition() const override;

private:
    int id_;
    double position_ = 0.0;
};

} // namespace robot
```

include/robot/unitree_motor.hpp

```
#pragma once

#include "robot/motor.hpp"
```

```
namespace robot {

class UnitreeMotor : public Motor {
public:
    explicit UnitreeMotor(int id);

    void enable() override;
    void setPosition(double position) override;
    double getPosition() const override;

private:
    int id_;
    double position_ = 0.0;
};

} // namespace robot
```

include/robot/robot.hpp

```
#pragma once

#include "robot/motor.hpp"

namespace robot {

class Robot {
public:
    Robot(Motor& left_motor, Motor& right_motor);

    void initialize();
    void move(double position);
    void printStatus() const;

private:
    Motor& left_motor_;
    Motor& right_motor_;
};

} // namespace robot
```

src/dm_motor.cpp

```
#include "robot/dm_motor.hpp"

#include <iomanip>
#include <iostream>

namespace robot {
```

```

DMMotor::DMMotor(int id)
    : id_(id)
{
}

void DMMotor::enable()
{
    std::cout << "DM Motor " << id_ << " enabled." << std::endl;
}

void DMMotor::setPosition(double position)
{
    position_ = position;
    std::cout << "DM Motor " << id_ << " -> target = " << std::fixed
                << std::setprecision(2) << position_ << " rad" << std::endl;
}

double DMMotor::getPosition() const
{
    return position_;
}

} // namespace robot

```

src/unitree_motor.cpp

```

#include "robot/unitree_motor.hpp"

#include <iomanip>
#include <iostream>

namespace robot {

UnitreeMotor::UnitreeMotor(int id)
    : id_(id)
{
}

void UnitreeMotor::enable()
{
    std::cout << "Unitree Motor " << id_ << " enabled." << std::endl;
}

void UnitreeMotor::setPosition(double position)
{
    position_ = position;
    std::cout << "Unitree Motor " << id_ << " -> target = " << std::fixed
                << std::setprecision(2) << position_ << " rad" << std::endl;
}

double UnitreeMotor::getPosition() const

```

```
{  
    return position_;  
}  
  
} // namespace robot
```

src/robot.cpp

```
#include "robot/robot.hpp"  
  
#include <iomanip>  
#include <iostream>  
  
namespace robot {  
  
Robot::Robot(Motor& left_motor, Motor& right_motor)  
    : left_motor_(left_motor), right_motor_(right_motor)  
{  
}  
  
void Robot::initialize()  
{  
    std::cout << "Robot initialization..." << std::endl << std::endl;  
    left_motor_.enable();  
    right_motor_.enable();  
}  
  
void Robot::move(double position)  
{  
    std::cout << std::endl << "Set robot target position: " << std::fixed  
        << std::setprecision(2) << position << " rad" << std::endl <<  
std::endl;  
    left_motor_.setPosition(position);  
    right_motor_.setPosition(position);  
    std::cout << std::endl << "Robot motion command finished." <<  
std::endl;  
}  
  
void Robot::printStatus() const  
{  
    std::cout << std::endl << "Left motor position: " << std::fixed  
        << std::setprecision(2) << left_motor_.getPosition() << "  
rad" << std::endl;  
    std::cout << "Right motor position: " << std::fixed  
        << std::setprecision(2) << right_motor_.getPosition() << "  
rad" << std::endl;  
}  
  
} // namespace robot
```

apps/main.cpp (最终入口文件)

```
#include "robot/dm_motor.hpp"
#include "robot/robot.hpp"
#include "robot/unitree_motor.hpp"

#include <iostream>

int main()
{
    std::cout << "=== Mini Robot Demo ===" << std::endl << std::endl;

    robot::DMMotor left_motor(1);
    robot::UnitreeMotor right_motor(2);
    robot::Robot robot(left_motor, right_motor);

    robot.initialize();
    robot.move(1.5);
    robot.printStatus();
    return 0;
}
```

最终验收：确认你的环境与代码全部正确

回到教程根目录并执行：

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
bash scripts/test_all.sh
```

正确时将看到：

```
[PASS] 01_hello_cpp
[PASS] 02_basic_motor
[PASS] 03_encapsulation
[PASS] 04_constructor
[PASS] 05_abstract_motor
[PASS] 06_inheritance
[PASS] 07_polymorphism
[PASS] 08_robot_composition
[PASS] mini_robot manual g++
[PASS] mini_robot CMake

All examples passed.
```

这表示 Ubuntu 开发环境、C++ 编译器、所有教学示例、多文件工程和 CMake 都已正常工作。如果某一项没有 PASS，请从该编号对应章节的目录、文件名和编译命令开始检查，而不要跳过错误继续下一步。

最后查看自己亲手建立的目录：

```
cd ~/robot_cpp_training/robot_cpp_oop_cmake_training
tree -L 3
```

简化后的预期结果：

```
robot_cpp_oop_cmake_training/
├── examples/
│   ├── 01_hello_cpp/
│   ├── 02_basic_motor/
│   ├── 03_encapsulation/
│   ├── 04_constructor/
│   ├── 05_abstract_motor/
│   ├── 06_inheritance/
│   ├── 07_polymorphism/
│   └── 08_robot_composition/
├── mini_robot/
│   ├── CMakeLists.txt
│   ├── apps/
│   ├── include/
│   ├── src/
│   ├── build/
│   └── build_test/
├── scripts/
└── test_all.sh
```

这个完整工程是你一步一步亲手建立起来的。